Email Testing Plan

Objective

Enable safe end-to-end email testing without sending emails to real recipients. All emails are redirected to a configurable **test email recipient** and clearly marked with `--TEST EMAIL FROM RFPO` in the subject line.

---

1. Test Mode Infrastructure

1.1 Admin Panel — Email Test Settings

Add a new **Email Test Mode** section to the existing email test tool page (`/tools/email-test`) with two fields:

Field	Type	Default	Description
Email Test Mode	Toggle (on/off)	ON	When enabled, all emails redirect to the test recipient
Test Email Recipient	Email input	`johnbouchard@icloud.com`	Single email address that receives ALL emails while test mode is active

1.2 Settings Storage — Database-Backed (Live Updates)

Note: `config/application_settings.json` exists but is **never loaded by any code**. For live toggle changes without container restarts, use a database-backed approach.

Add two rows to the existing `list` table (key-value store) or create a lightweight `email_test_settings` singleton:

```
# Option: Use existing List model as key-value store
# key="email_test_mode", value="true"
# key="email_test_recipient", value="johnbouchard@icloud.com"
```

Admin UI reads/writes directly to the DB. `send_email()` queries on each call with a short TTL cache (5 seconds) to avoid hammering the DB on burst sends.

1.3 EmailService Interception Layer

Modify `email_service.py` `send_email()` to intercept **at the top, before any retry/queue logic**:

- If **test mode is ON**:
    1. **Snapshot** test settings at method entry (prevents race conditions if admin toggles mid-send)
    2. Prepend `--TEST EMAIL FROM RFPO` to the subject line
    3. Replace **all recipients** (TO, CC, BCC) with the test recipient address
    4. Prepend an info block to both `body_html` and `body_text` showing original recipients

5. Keep `from_email` unchanged (ACS/SMTP sender must remain valid for delivery)
6. Preserve any attachments as-is
7. Log the redirect: `"TEST MODE: Redirected email for [original@email.com] → [test@email.com]"`

- If **test mode is OFF**: Normal behavior, no changes

Retry queue safety: Because transformation happens before `_enqueue_failed()`, retried emails already have the correct test subject/recipients — no double-prefixing.

1.4 Original Recipients Info Block

Prepended to the **top** of the email body (before the main content) in both HTML and plain text:

HTML version:

```
<div style="background:#fff3cd;border:2px solid #ffc107;border-radius:8px;padding:16px;margin-bottom:24px;font-family:Arial,sans-serif;">
  <strong>⚠ TEST MODE — Email Redirected</strong><br>
  <strong>Original TO:</strong> approver1@company.com, approver2@company.com<br>
  <strong>Original CC:</strong> finance@company.com<br>
  <strong>Original BCC:</strong> admin@company.com<br>
</div>
```

Plain text version:

```
═══════════════════════════════════════
⚠  TEST MODE — Email Redirected
═══════════════════════════════════════
Original TO:  approver1@company.com, approver2@company.com
Original CC:  finance@company.com
Original BCC: admin@company.com
═══════════════════════════════════════
```

---

2. Email Trigger Inventory & Test Matrix

2.1 Currently Functional (5 triggers)

#	Email Type	Trigger	Intended Recipient	Template	Location
1	**Welcome Email**	New user created in admin panel	The new user	`welcome.html`	`custom_admin.py` ~L2725

#	Email Type	Trigger	Intended Recipient	Template	Location
2	**Password Changed**	User changes password	The user	`password_changed.html`	`simple_api.py` ~L673
3	**Approval: Submit**	RFPO submitted for approval	First-stage approver(s)	`approval_notification.html`	`simple_api.py` ~L1307
4	**Approval: Complete**	Final approval/refusal	RFPO requestor (creator)	`approval_notification.html`	`simple_api.py` ~L976
5	**Approval: Reassign**	Approval action reassigned	New approver	`approval_notification.html`	`simple_api.py` ~L1478

2.2 Defined But Not Wired (3 triggers)

#	Email Type	Status	Notes
6	**User Added to Project**	Function exists, zero call sites	Template: `user_added_to_project.html`
7	**Approval: Stage Advance**	No notification hook in `advance_to_next_step()`	Known gap (Issue #06) — approvers at stage 2+ never notified when workflow advances to their stage
8	**Approval Complete (alt template)**	Template `approval_complete.html` exists but unused	`approval_notification.html` is used instead for completion notifications

2.3 Edge Cases in Approval Workflow

Case	Description	Email Behavior
Multi-approver first stage	First stage has multiple parallel approvers	Each approver gets a separate email (loop in simple_api.py ~L1307)
Conditional approval	Approver marks "conditional" instead of "approved"	Same template as approval, subject includes approval type
Multi-stage workflow	Workflow has stages 1→2→3	❌ **GAP:** After stage 1 completes, stage 2 approvers get NO notification
Timeout/overdue	Approval sits past SLA	❌ **No timeout system exists** — no overdue notifications

2.4 Consortium Email Fields (defined but unused)

- `Consortium.doc_email_address` — intended for required-documents contact
- `Consortium.doc_email_name` — contact name for required docs
- `Consortium.po_email` — intended for completed PO delivery

- These exist in the model but are never referenced in email-sending code. Future use TBD.

---

3. Test Execution Plan

Phase 1: Implement Test Mode

1. Add database-backed settings for `email_test_mode` and `email_test_recipient`
2. Add admin UI controls on the email test tool page
3. Implement interception in `EmailService.send_email()` with settings snapshot
4. Add persistent warning banner to admin panel when test mode is active
5. Deploy to Azure (admin + API containers)

Phase 2: Validate Each Email Trigger

Execute each test in order, verifying the test email arrives at `johnbouchard@icloud.com`:

Test	Action	Expected Subject	Verify
T1	Create a test user in admin panel	`--TEST EMAIL FROM RFPO Welcome to RFPO Application...`	Login link, temp password, original recipient in info block
T2	Change password via user app	`--TEST EMAIL FROM RFPO Password Changed - RFPO Application...`	Change timestamp, IP address, original recipient
T3	Submit an RFPO for approval	`--TEST EMAIL FROM RFPO RFPO Approval Required - [ID]`	RFPO ID, approval type, link to RFPO, all original approvers listed
T4	Approve/refuse RFPO (final step)	`--TEST EMAIL FROM RFPO RFPO Approval Required - RFPO Approved`	Outcome, requestor shown as original recipient
T5	Reassign an approval action	`--TEST EMAIL FROM RFPO Reassigned: [step name]`	New approver shown as original recipient
T6	Use existing email test tool	`--TEST EMAIL FROM RFPO [custom subject]`	Test tool still works, redirected to test recipient

Phase 3: Review Results

- Verify all 6 test emails arrive at test recipient
- Verify original recipients shown in info block (not in actual email headers)
- Verify `--TEST EMAIL FROM RFPO` prefix on all subjects
- Verify body content is complete and correct per template
- Verify attachments preserved (if any future use)
- Document any failures

---

4. Considerations

Security

- Test mode setting stored server-side in DB — not exposable via client API
- Test recipient must be validated as a proper email format before saving
- Warn in admin UI when test mode is active (persistent banner)

Operational Safety

- **Persistent yellow banner** across ALL admin pages when test mode is ON: `⚠ Email Test Mode Active — all emails redirected to johnbouchard@icloud.com`
- Log all test-mode redirections with original recipients for audit trail
- When turning OFF test mode, display confirmation: "Emails will now be sent to real recipients"

Email Provider

- Current provider: Azure Communication Services (ACS) with SMTP fallback
- Test mode works at the application layer **before** provider selection — no ACS/SMTP changes needed
- `from_email` left unchanged so ACS sender validation passes

Retry Queue

- Test mode transformation applied BEFORE `_enqueue_failed()` stores the email
- Retried emails already have test subject/recipients — no double-prefixing risk
- Failed queue snapshot (admin diagnostics) will show test recipient, not originals

Race Conditions

- Settings snapshotted once at `send_email()` entry point
- If admin disables test mode while a send is in progress, that send completes in test mode
- Next send reads fresh settings

What This Does NOT Test

- Email deliverability/spam scoring to actual recipient domains
- Template rendering across email clients (Gmail, Outlook, Apple Mail)
- ACS/SMTP failover (already handled by retry queue)
- Approval stage-advance notifications (not wired — Issue #06)

---

5. Files to Modify

File	Change
`models.py`	Add `EmailTestSettings` model (or use `List` model as key-value store)
`email_service.py`	Add test-mode interception at top of `send_email()`, settings query with TTL cache
`custom_admin.py`	Add admin route + UI for email test settings on email test tool page

File	Change
`templates/admin/tools/email_test.html`	Add test mode toggle + recipient field to existing email test page
`templates/admin/base.html`	Add persistent warning banner when test mode is active
`simple_api.py`	(No changes — interception is in email_service.py)

Migration

- `db.create_all()` will create the new table (safe — doesn't affect existing tables)
- Initial values set by seed script or first admin save: `test_mode=True`, `recipient=johnbouchard@icloud.com`